

Question 1: Real-Time Face Detection and Recognition Pipeline

1. Problem Understanding (Inputs / Outputs / Constraints)

The system must continuously acquire frames from a live camera, detect all faces present, recognize each against a pre-trained dataset of known identities, and display bounding boxes with labels — all while the stream keeps running.

Constraints identified:

- **Continuous operation** — the system must run in an unbroken loop for as long as the video source is active, without resource leaks over time.
- **Minimized processing delay** — each iteration of the loop must complete quickly enough to keep the displayed video feeling live.
- **Multiple faces** — detection and recognition must handle zero, one, or several faces present in the same frame, not just a single face.

2. Algorithm Design Overview

The pipeline is split into independent, single-responsibility modules — `acquire_frame`, `preprocess`, `detect_faces`, `recognize_face`, and `render` — connected by a thin main loop that only orchestrates calls and never contains business logic itself. This modularity directly supports the "handle multiple faces" constraint, since `detect_faces` naturally returns a list, and everything downstream (`recognize_face`, `render`) is written to iterate over that list rather than assume a single face. The continuous-operation and low-delay constraints are addressed at the loop level: a `while True` structure with an explicit exit condition keeps the system running indefinitely, while resource acquisition (camera handle, recognizer model) happens once *before* the loop, not inside it, so per-frame cost is limited to genuinely per-frame work.

3. Modular Pseudocode

3.1 Initialization (runs once)

```
FUNCTION main():
    camera          = OPEN_VIDEO_CAPTURE(source = 0)
    face_detector    = LOAD_FACE_DETECTOR(model = "haar_cascade" OR "dnn_ssd")
    face_recognizer  = LOAD_RECOGNIZER(model_path, trained_on = known_faces_dataset)
    known_labels     = LOAD_LABEL_MAP(known_faces_dataset)

    IF camera NOT OPENED:
        LOG_ERROR("Camera unavailable")
        RETURN

    RUN_MAIN_LOOP(camera, face_detector, face_recognizer, known_labels)
    CLOSE_VIDEO_CAPTURE(camera)
    DESTROY_ALL_WINDOWS()
END FUNCTION
```

3.2 Main Loop (continuous, low-delay)

```
FUNCTION RUN_MAIN_LOOP(camera, face_detector, face_recognizer, known_labels):
    WHILE camera IS OPEN:
        frame_ok, frame = ACQUIRE_FRAME(camera)
        IF NOT frame_ok:
            CONTINUE                # skip malformed frame, keep loop alive

        processed_frame = PREPROCESS(frame)

        face_boxes = DETECT_FACES(processed_frame, face_detector)

        results = []                # supports MULTIPLE faces
        FOR EACH box IN face_boxes:
            face_crop = CROP(processed_frame, box)
            label, confidence = RECOGNIZE_FACE(face_crop, face_recognizer, known_labels)
            APPEND results, (box, label, confidence)
        END FOR
```

```

        output_frame = RENDER(frame, results)
        DISPLAY(output_frame)

        key = WAIT_KEY(timeout = 1)          # non-blocking, bounds loop delay
        IF key == 'q':
            BREAK                             # controlled exit from continuous loop
    END WHILE
END FUNCTION

```

3.3 Acquisition & Pre-processing Modules

```

FUNCTION ACQUIRE_FRAME(camera):
    success, frame = camera.READ()
    RETURN success, frame
END FUNCTION

FUNCTION PREPROCESS(frame):
    gray = CONVERT_COLOR(frame, BGR_TO_GRAY)
    gray = APPLY_CLAHE(gray)                # normalizes lighting
    gray = GAUSSIAN_BLUR(gray, kernel = 3x3)
    RETURN gray
END FUNCTION

```

3.4 Detection Module

```

FUNCTION DETECT_FACES(processed_frame, face_detector):
    boxes = face_detector.DETECT_MULTI_SCALE(
        processed_frame,
        scale_factor = 1.1,
        min_neighbors = 5,
        min_size = (40, 40))

    RETURN boxes                                # list of (x, y, w, h), possibly empty, one, or many
END FUNCTION

```

3.5 Recognition Module

```

FUNCTION RECOGNIZE_FACE(face_crop, face_recognizer, known_labels):
    face_crop = RESIZE(face_crop, target_size)
    embedding = face_recognizer.COMPUTE_EMBEDDING(face_crop)
    label_id, distance = NEAREST_NEIGHBOR_MATCH(embedding, known_labels)

    IF distance > THRESHOLD:
        RETURN "Unknown", distance
    ELSE:
        RETURN known_labels[label_id], distance
    END IF
END FUNCTION

```

3.6 Display Module

```

FUNCTION RENDER(frame, results):
    FOR EACH (box, label, confidence) IN results:
        DRAW_RECTANGLE(frame, box, color = GREEN, thickness = 2)
        DRAW_TEXT(frame, label + " (" + confidence + ")",
            position = ABOVE(box), color = GREEN)
    END FOR
    RETURN frame
END FUNCTION

```

4. Use of Control Structures

The design uses a single top-level WHILE camera IS OPEN loop for continuous operation, with an explicit BREAK exit condition tied to user input rather than an implicit or missing termination path. Inside each iteration, a FOR EACH loop over face_boxes handles the variable-count "multiple faces" requirement without any hard-coded face limit. An early CONTINUE guards against malformed frames without aborting the whole session, and a non-blocking WAIT_KEY(timeout = 1) (rather than an unbounded wait) ensures the loop's own polling never becomes the delay bottleneck. Functions are used throughout instead of inline code so each stage (acquisition, preprocessing, detection, recognition, display) can be modified, benchmarked, or swapped independently.

5. Completeness & Efficiency Notes

The pipeline explicitly handles the zero-face case (empty results list — loop simply displays the raw frame), the single-face case, and the multi-face case identically, since every downstream step operates on a list. Unrecognised faces are handled via a distance threshold that yields an "Unknown" label rather than crashing or mis-assigning a known identity. Efficiency is preserved by loading the camera and models once outside the loop, by preprocessing only what detection/recognition need (grayscale, not full color pipeline), and by keeping per-frame work proportional to the number of faces actually found rather than a fixed worst case.

Question 2: OCR Pipeline for Document Processing

1. Problem Understanding (Inputs / Outputs / Constraints)

The system must read a batch of (potentially noisy, low-contrast) document images, enhance them for legibility, segment them into text regions, and extract/output the recognized text for each file.

Constraints identified:

- **Noisy / low-contrast input** — source images may be scanned unevenly, have shadows, or low text-to-background contrast.
- **Batch processing** — the system must process a collection of images (not a single one) reliably and report per-file results.

2. Algorithm Design Overview

The pipeline is organised as a batch driver (process_batch) that calls a per-image pipeline (process_single_document) composed of four independent stages — enhance_image, segment_text_regions, extract_text, and result reporting. Separating the batch driver from the single-image pipeline means the noisy-input handling logic lives entirely inside enhance_image and the batch-processing requirement lives entirely inside process_batch — the two constraints are solved by two different, independently testable modules rather than being tangled together. Each stage returns a well-defined output that the next stage consumes (raw image → enhanced image → list of text-region crops → list of recognized strings), which keeps the pipeline easy to reason about and to re-order or replace stages in.

3. Modular Pseudocode

3.1 Batch Driver

```
FUNCTION process_batch(input_folder, output_folder):
    image_paths = LIST_FILES(input_folder, extensions = [".png", ".jpg", ".jpeg", ".tiff"])
    batch_report = []

    FOR EACH path IN image_paths:
        TRY:
            text_result = process_single_document(path)
            SAVE_TEXT(output_folder, FILENAME(path), text_result)
            APPEND batch_report, (path, "SUCCESS", LENGTH(text_result))
        CATCH error:
            LOG_ERROR("Failed on " + path + ": " + error)
            APPEND batch_report, (path, "FAILED", error)
        END TRY
    END FOR

    WRITE_REPORT(output_folder, batch_report) # per-file batch summary
    RETURN batch_report
END FUNCTION
```

3.2 Single-Document Pipeline

```
FUNCTION process_single_document(image_path):
    raw_image = READ_IMAGE(image_path)
    enhanced_image = enhance_image(raw_image)
    regions = segment_text_regions(enhanced_image)
    extracted_text = extract_text(enhanced_image, regions)
    RETURN extracted_text
END FUNCTION
```

3.3 Enhancement Module (handles noisy / low-contrast input)

```
FUNCTION enhance_image(image):
    gray = CONVERT_COLOR(image, BGR_TO_GRAY)
    denoised = FAST_NL_MEANS_DENOISE(gray) # removes sensor / scan noise
    contrast = APPLY_CLAHE(denoised) # boosts local text contrast
    binary = ADAPTIVE_THRESHOLD(contrast,
                                method = GAUSSIAN, block_size = 31, C = 10)
    deskewed = DESKEW(binary) # corrects scan-angle tilt
    RETURN deskewed
```

```
END FUNCTION
```

3.4 Segmentation Module

```
FUNCTION segment_text_regions(enhanced_image):
    dilated = MORPHOLOGICAL_DILATE(enhanced_image, kernel = horizontal_line)
    contours = FIND_CONTOURS(dilated)

    regions = []
    FOR EACH contour IN contours:
        box = BOUNDING_RECT(contour)
        IF box.width > MIN_WIDTH AND box.height > MIN_HEIGHT:
            APPEND regions, box
        END IF
    END FOR

    regions = SORT(regions, order = "top_to_bottom_then_left_to_right")
    RETURN regions
END FUNCTION
```

3.5 Text Extraction Module

```
FUNCTION extract_text(enhanced_image, regions):
    full_text = ""
    FOR EACH region IN regions:
        crop = CROP(enhanced_image, region)
        crop = RESIZE_FOR_OCR(crop)
        line_text = OCR_ENGINE.RECOGNIZE(crop)          # e.g. Tesseract call
        full_text = full_text + line_text + NEWLINE
    END FOR
    RETURN full_text
END FUNCTION
```

4. Use of Control Structures

A top-level FOR EACH path IN image_paths loop implements batch processing, wrapped in a TRY/CATCH so a single corrupt or unreadable file logs an error and is skipped, rather than aborting the entire batch — this is essential for batch robustness. Inside the single-document pipeline, sequential function calls (rather than one monolithic block) make each enhancement/segmentation/extraction stage independently swappable. A nested FOR EACH contour loop with a size-filter IF condition removes spurious small contours (noise artefacts) before they reach the (comparatively expensive) OCR step, and a second FOR EACH region loop drives the actual text extraction in reading order.

5. Completeness & Efficiency Notes

Noisy/low-contrast input is handled by an explicit, ordered enhancement chain — denoise before contrast-boost before thresholding before deskew — so each step's assumptions are met by the previous step's output. Batch processing is handled completely: every file in the input folder is attempted, failures are caught and logged individually rather than halting the run, and a consolidated report is written so the user can see which documents succeeded or failed without re-running the whole batch. Region sorting before OCR ensures multi-column or multi-line documents are read in a sensible order rather than contour-discovery order.

Question 5: Facial Expression Recognition System (Mobile)

1. Problem Understanding (Inputs / Outputs / Constraints)

The system must continuously capture frames from a phone camera, detect the user's face, extract lightweight features, classify the expression (happy/sad/neutral), and display the result — while staying within a mobile CPU's compute budget.

Constraints identified:

- **Limited computational resources** — must run on a mobile CPU without heavy per-frame cost.
- **Real-time response** — expression output must update with minimal perceptible lag as the user's face changes.

2. Algorithm Design Overview

The pipeline mirrors Question 1's modular structure (acquire → detect → extract → classify → display) but is intentionally lighter at every stage, since both constraints here point the same direction: keep per-frame cost small. The key design decision is restricting detection/classification to a single primary face (the phone's user) with a tracked region-of-interest, rather than the multi-face search used in Question 1 — full re-detection only happens periodically, and a lightweight feature extractor plus a small classifier is used instead of a deep CNN. A short result-smoothing buffer is added as its own module so that real-time responsiveness does not come at the cost of flickering, unstable output.

3. Modular Pseudocode

3.1 Initialization

```
FUNCTION main():
    camera          = OPEN_VIDEO_CAPTURE(source = 0)
    face_detector    = LOAD_FACE_DETECTOR(model = "haar_cascade")
    expression_model = LOAD_LIGHTWEIGHT_CLASSIFIER(model = "small_cnn" OR "svm_on_lbp")
    recent_predictions = EMPTY_QUEUE(max_size = 6)    # for smoothing

    RUN_MAIN_LOOP(camera, face_detector, expression_model, recent_predictions)
    CLOSE_VIDEO_CAPTURE(camera)
END FUNCTION
```

3.2 Main Loop (efficient, real-time)

```
FUNCTION RUN_MAIN_LOOP(camera, face_detector, expression_model, recent_predictions):
    last_known_box = NULL
    frame_count     = 0

    WHILE camera IS OPEN:
        frame_ok, frame = ACQUIRE_FRAME(camera)
        IF NOT frame_ok:
            CONTINUE

        frame_count = frame_count + 1

        IF last_known_box IS NULL OR frame_count MOD 5 == 0:
            last_known_box = DETECT_PRIMARY_FACE(frame, face_detector) # full re-detect periodically
        END IF

        IF last_known_box IS NOT NULL:
            face_crop = CROP(frame, last_known_box)
            features   = EXTRACT_FEATURES(face_crop)
            raw_label  = CLASSIFY_EXPRESSION(features, expression_model)

            PUSH(recent_predictions, raw_label)
            stable_label = SMOOTH(recent_predictions)

            output_frame = RENDER(frame, last_known_box, stable_label)
        ELSE:
            output_frame = frame
        END IF
```

```

        DISPLAY(output_frame)

        key = WAIT_KEY(timeout = 1)
        IF key == 'q':
            BREAK
    END WHILE
END FUNCTION

```

3.3 Detection Module

```

FUNCTION DETECT_PRIMARY_FACE(frame, face_detector):
    gray = CONVERT_COLOR(frame, BGR_TO_GRAY)
    gray = APPLY_CLAHE(gray)
    boxes = face_detector.DETECT_MULTI_SCALE(gray, scale_factor = 1.15, min_neighbors = 4)

    IF LENGTH(boxes) == 0:
        RETURN NULL
    ELSE:
        RETURN LARGEST(boxes)          # assume closest/primary face
    END IF
END FUNCTION

```

3.4 Feature Extraction Module

```

FUNCTION EXTRACT_FEATURES(face_crop):
    aligned = ALIGN_FACE(face_crop)          # corrects small head tilt
    resized = RESIZE(aligned, 48x48)
    gray = CONVERT_COLOR(resized, BGR_TO_GRAY)
    features = COMPUTE_LBP_HISTOGRAM(gray)    # cheap, mobile-friendly descriptor
    RETURN features
END FUNCTION

```

3.5 Classification Module

```

FUNCTION CLASSIFY_EXPRESSION(features, expression_model):
    label = expression_model.PREDICT(features) # returns "happy" | "sad" | "neutral"
    RETURN label
END FUNCTION

```

3.6 Smoothing & Display Modules

```

FUNCTION SMOOTH(recent_predictions):
    RETURN MAJORITY_VOTE(recent_predictions)
END FUNCTION

FUNCTION RENDER(frame, box, label):
    DRAW_RECTANGLE(frame, box, color = BLUE)
    DRAW_TEXT(frame, label, position = ABOVE(box))
    RETURN frame
END FUNCTION

```

4. Use of Control Structures

The main loop uses an IF `last_known_box IS NULL OR frame_count MOD 5 == 0` condition to control *when* the relatively expensive full detection step runs, rather than running it every iteration — this is the central control-structure decision that satisfies the limited-resources constraint. A fixed-size queue (`PUSH/EMPTY_QUEUE(max_size=6)`) implements the smoothing buffer without unbounded memory growth. The overall loop remains a single WHILE `camera IS OPEN` with an early CONTINUE for bad frames and a controlled BREAK for exit, matching the real-time constraint by never letting any one stage block indefinitely.

5. Completeness & Efficiency Notes

The pipeline handles the no-face-detected case explicitly (falls back to displaying the raw frame rather than crashing on a NULL box), reuses the last known face location between full re-detections to avoid redundant expensive detection calls, and always classifies from a stabilised, majority-voted label rather than a single noisy prediction. Because feature extraction (LBP) and classification (small model) are both deliberately lightweight, the pipeline meets the mobile-CPU constraint without needing frame-skipping on the classification step itself — only detection is throttled, since it is the most expensive stage.

Question 6: Real-Time Object Detection Pipeline

1. Problem Understanding (Inputs / Outputs / Constraints)

The system must continuously capture video frames, run a trained detector to find all objects present (of possibly several classes), draw bounding boxes and labels for each, and display the result live.

Constraints identified:

- **Efficient frame processing** — each frame must be handled quickly enough to sustain a live video loop.
- **Multiple objects** — the system must detect and display an arbitrary number of objects per frame, of possibly different classes.

2. Algorithm Design Overview

The pipeline follows the same acquire → preprocess → detect → visualize → display shape used throughout, with the detection module explicitly designed around a trained multi-class, multi-instance detector (e.g. an SSD/YOLO-style model) rather than a single-object detector, so that "handle multiple objects" is a native property of the detection module's output type (a list of detections) rather than something bolted on afterward. Efficiency is achieved by keeping the detection model's input size small and fixed, and by applying non-maximum suppression once per frame to collapse duplicate/overlapping boxes before they reach the (cheap) visualization stage.

3. Modular Pseudocode

3.1 Initialization

```
FUNCTION main():
    camera      = OPEN_VIDEO_CAPTURE(source = 0)
    object_model = LOAD_TRAINED_DETECTOR(model = "ssd_mobilenet" OR "yolo_tiny")
    class_labels = LOAD_CLASS_LABELS(label_file)

    RUN_MAIN_LOOP(camera, object_model, class_labels)
    CLOSE_VIDEO_CAPTURE(camera)
END FUNCTION
```

3.2 Main Loop (efficient, multi-object)

```
FUNCTION RUN_MAIN_LOOP(camera, object_model, class_labels):
    WHILE camera IS OPEN:
        frame_ok, frame = ACQUIRE_FRAME(camera)
        IF NOT frame_ok:
            CONTINUE

        input_blob      = PREPROCESS(frame, target_size = model_input_size)
        raw_detections  = DETECT_OBJECTS(input_blob, object_model)
        detections       = FILTER_AND_SUPPRESS(raw_detections, conf_threshold = 0.5)

        output_frame = RENDER(frame, detections, class_labels)
        DISPLAY(output_frame)

        key = WAIT_KEY(timeout = 1)
        IF key == 'q':
            BREAK
    END WHILE
END FUNCTION
```

3.3 Pre-processing Module

```
FUNCTION PREPROCESS(frame, target_size):
    resized      = RESIZE(frame, target_size)
    normalized   = NORMALIZE_PIXELS(resized, mean, scale)
    blob         = TO_MODEL_INPUT_FORMAT(normalized)
    RETURN blob
END FUNCTION
```

3.4 Detection Module

```
FUNCTION DETECT_OBJECTS(input_blob, object_model):
```

```

object_model.SET_INPUT(input_blob)
raw_output = object_model.FORWARD_PASS()

detections = []
FOR EACH prediction IN raw_output:
    IF prediction.confidence > MIN_CONFIDENCE:
        APPEND detections, (prediction.box, prediction.class_id, prediction.confidence)
    END IF
END FOR
RETURN detections                                # supports MULTIPLE objects natively
END FUNCTION

```

3.5 Non-Maximum Suppression Module

```

FUNCTION FILTER_AND_SUPPRESS(detections, conf_threshold):
    boxes, scores, class_ids = SPLIT_FIELDS(detections)
    keep_indices = NON_MAX_SUPPRESSION(boxes, scores,
                                       score_threshold = conf_threshold, iou_threshold = 0.45)

    filtered = []
    FOR EACH index IN keep_indices:
        APPEND filtered, detections[index]
    END FOR
    RETURN filtered
END FUNCTION

```

3.6 Visualization Module

```

FUNCTION RENDER(frame, detections, class_labels):
    FOR EACH (box, class_id, confidence) IN detections:
        label_text = class_labels[class_id] + " " + ROUND(confidence, 2)
        DRAW_RECTANGLE(frame, box, color = COLOR_FOR_CLASS(class_id), thickness = 2)
        DRAW_TEXT(frame, label_text, position = ABOVE(box))
    END FOR
    RETURN frame
END FUNCTION

```

4. Use of Control Structures

The outer WHILE camera IS OPEN loop again provides continuous, real-time operation with an early CONTINUE for bad frames and a user-driven BREAK for exit. Two nested FOR EACH loops handle the variable-count multi-object requirement: one iterates raw model outputs to filter by confidence, and a second iterates the NMS-kept indices to draw results — both scale naturally to however many objects the model finds, with no fixed upper bound coded in. Separating DETECT_OBJECTS (raw inference) from FILTER_AND_SUPPRESS (NMS/thresholding) keeps the expensive forward pass isolated from the cheap post-processing logic, which is the main structural decision supporting per-frame efficiency.

5. Completeness & Efficiency Notes

The pipeline explicitly supports zero, one, or many detected objects per frame through list-based data flow, and multiple object *classes* simultaneously via per-detection class_id lookups rather than assuming a single object type. Non-maximum suppression prevents the same physical object being drawn multiple times as overlapping boxes, which would otherwise misrepresent the count/visual result. Efficiency is preserved by fixing the model's input size (bounding preprocessing/inference cost) and by doing confidence filtering before, not after, the more expensive NMS step, reducing the number of boxes NMS has to consider.

Question 13: Video Stabilization Pipeline

1. Problem Understanding (Inputs / Outputs / Constraints)

The system must read video frames sequentially, estimate the motion between consecutive frames, and apply a corrective transform so the displayed output appears stabilized (reduced jitter) while still running in real time.

Constraints identified:

- **Real-time processing** — motion estimation and correction must keep up with the incoming frame rate.
- **Jitter reduction** — the output must visibly reduce shaky/unintended camera motion while preserving intentional camera movement (e.g. panning).

2. Algorithm Design Overview

Unlike the detection-style pipelines above, stabilization is inherently a *temporal* problem: each output frame depends on a short history of previous frames' motion, not just the current frame in isolation. The design therefore introduces a persistent, growing/rolling cumulative_trajectory state carried across loop iterations, alongside the usual acquire/process/display modules. The pipeline is split into estimate_motion (frame-to-frame transform), update_trajectory (accumulate and smooth the camera's path over time), and apply_stabilizing_transform (warp the current frame to cancel the unwanted, non-smooth component of that path) — separating *measuring* motion from *correcting* for it keeps the smoothing strategy (constraint: jitter reduction) independent of the motion-estimation method (constraint: real-time performance), so either can be tuned/replaced without touching the other.

3. Modular Pseudocode

3.1 Initialization

```
FUNCTION main():
    video = OPEN_VIDEO_CAPTURE(source_path)
    frame_ok, previous_frame = ACQUIRE_FRAME(video)
    previous_gray = CONVERT_COLOR(previous_frame, BGR_TO_GRAY)

    cumulative_trajectory = ZERO_VECTOR(dx = 0, dy = 0, d_angle = 0)
    smoothing_window      = EMPTY_QUEUE(max_size = 30)

    RUN_MAIN_LOOP(video, previous_frame, previous_gray, cumulative_trajectory, smoothing_window)
    CLOSE_VIDEO_CAPTURE(video)
END FUNCTION
```

3.2 Main Loop (real-time, jitter-aware)

```
FUNCTION RUN_MAIN_LOOP(video, previous_frame, previous_gray, cumulative_trajectory, smoothing_window):
    WHILE video HAS_NEXT_FRAME:
        frame_ok, current_frame = ACQUIRE_FRAME(video)
        IF NOT frame_ok:
            BREAK # end of stream
        END IF

        current_gray = CONVERT_COLOR(current_frame, BGR_TO_GRAY)

        raw_motion = estimate_motion(previous_gray, current_gray)

        cumulative_trajectory = update_trajectory(cumulative_trajectory, raw_motion)
        PUSH(smoothing_window, cumulative_trajectory)
        smoothed_trajectory = SMOOTH_TRAJECTORY(smoothing_window)

        correction = smoothed_trajectory - cumulative_trajectory
        stabilized_frame = apply_stabilizing_transform(current_frame, correction)

        DISPLAY(stabilized_frame)

        previous_gray = current_gray
        previous_frame = current_frame

    key = WAIT_KEY(timeout = 1)
```

```

        IF key == 'q':
            BREAK
    END WHILE
END FUNCTION

```

3.3 Motion Estimation Module

```

FUNCTION estimate_motion(previous_gray, current_gray):
    prev_points = DETECT_GOOD_FEATURES_TO_TRACK(previous_gray, max_corners = 200)
    curr_points, status = CALC_OPTICAL_FLOW_PYR_LK(previous_gray, current_gray, prev_points)

    valid_prev = FILTER(prev_points, status == FOUND)
    valid_curr = FILTER(curr_points, status == FOUND)

    transform = ESTIMATE_RIGID_TRANSFORM(valid_prev, valid_curr) # dx, dy, d_angle
    RETURN transform
END FUNCTION

```

3.4 Trajectory Update & Smoothing Modules

```

FUNCTION update_trajectory(cumulative_trajectory, raw_motion):
    RETURN cumulative_trajectory + raw_motion # running sum of frame-to-frame motion
END FUNCTION

FUNCTION SMOOTH_TRAJECTORY(smoothing_window):
    RETURN MOVING_AVERAGE(smoothing_window) # low-pass filter over recent path
END FUNCTION

```

3.5 Stabilizing Transform Module

```

FUNCTION apply_stabilizing_transform(current_frame, correction):
    transform_matrix = BUILD_AFFINE_MATRIX(dx = correction.dx,
                                           dy = correction.dy,
                                           angle = correction.d_angle)

    stabilized = WARP_AFFINE(current_frame, transform_matrix)
    stabilized = CROP_TO_REMOVE_BORDER_ARTIFACTS(stabilized, margin_ratio = 0.05)
    RETURN stabilized
END FUNCTION

```

4. Use of Control Structures

The main WHILE video HAS_NEXT_FRAME loop carries state (previous_gray, cumulative_trajectory, smoothing_window) across iterations by explicit reassignment at the end of each pass — this is the key control-structure feature that makes temporal stabilization possible, distinguishing it from the stateless per-frame loops in the other pipelines. A fixed-size sliding-window queue (PUSH into smoothing_window) bounds memory while still supporting the moving-average smoothing needed for jitter reduction. A guarded BREAK on end-of-stream (NOT frame_ok) cleanly terminates the loop instead of looping on stale/empty frames, keeping the system real-time-safe even near the end of a finite video file.

5. Completeness & Efficiency Notes

The pipeline separates raw per-frame motion (raw_motion) from the smoothed intended path (smoothed_trajectory), so the correction applied is exactly the high-frequency jitter component, not the camera's genuine intentional movement (e.g. an intentional pan) — this directly satisfies the jitter-reduction constraint without freezing the whole shot. Real-time performance is supported by using sparse optical flow on a bounded number of tracked feature points (max_corners = 200) rather than dense per-pixel motion estimation, which would be far more expensive. Border-artifact cropping after warping is included so the stabilized output does not expose undefined edge pixels introduced by the correcting transform, completing the pipeline from raw input to a clean, displayable stabilized frame.
